CS413 · MIDTERM 1 · OCTOBER 2026

From extended LAMBDA to three pure forms.

A compositional compiler, Church encodings,
and one call-by-value evaluation engine.

Bijan Nossoughi

Compile syntax → reduce pure terms → observe data

Rich source. Minimal target.

Source AST

Naturals, booleans, arithmetic, comparisons, pairs, conditionals, functions, let, and recursion.

Uses the supplied Assignment 5 visitor interface unchanged.

Target AST

```
Var(name)
Lam(name, body)
App(function, argument)
```

No primitive numbers, operators, pairs, or unresolved encoding names.

Natural subtraction truncates at zero.
Conditionals test booleans; and / or short-circuit.

One evaluator; explicit module boundaries.

Source AST → Compiler visitor + Church builders → Pure target AST

Shared evaluation machine

Accepts either AST.
Common application frames enforce function → argument → application.

Observation layer

Builds pure target contexts.
Calls the same evaluator.
Inspects final inert data shapes.

Compiler never calls evaluation. Target reduction never calls source arithmetic.

Data becomes behavior.

Data	Pure lambda representation
True / false	`λt. λf. t  /  λt. λf. f`
Natural 2	`λs. λz. s (s z)`
Pair	`λa. λb. λp. p a b`
Identity	`I = λw. w`

> A boolean chooses. A numeral iterates. A pair supplies two values to a selector.

Weak, left-to-right call-by-value.

```
(λx. body) value  →  body[x := value]

Target value = lambda abstraction
```

Application order

1. Evaluate the function.
2. Evaluate the argument.
3. Substitute and count one beta step.

Weak evaluation

Stop at a lambda value.
Do not reduce under its binder.
Do not perform eta reduction.

The budget boundary is part of correctness.

Input	Budget	Outcome	Beta steps
`I`	0	Completed value	0
`I I`	0	Exhausted	0
`I I`	1	Completed value	1

> Check the budget immediately before contraction.
> A value on the final permitted step still completes.

Exhaustion reports insufficient resources. It does not prove nontermination.

Fresh names prevent accidental capture.

Substitution hazard

```
(λy. x)[x := y]

Wrong:   λy. y
Correct: λy_1. y
```

Per-call naming supply

Reserve all source names.
Reserve names in translated terms.
Choose the first unused suffix.

Deterministic; no global counters.

```
FV(C[e]) = FV(e)
```

Let binds only its body. If fix binders coincide, the parameter shadows the function name.

Translate each constructor locally.

```
C[x]                  = x
C[λx. e]              = λx. C[e]
C[e1 e2]              = C[e1] C[e2]
C[let x = e1 in e2] = (λx. C[e2]) C[e1]
```

Let's argument position preserves strict initializer evaluation.
Its lambda binds only the body.

Every child is checked, including unused branches and unapplied bodies.
Compiling LOOP returns syntax without executing it.

Delay both branches. Force the selected one.

```
C[if c then t else f] =
  (C[c] (λu. C[t]) (λv. C[f])) I
```

Why thunks?

Branch lambdas are already values.
The Church boolean selects one.
Applying I forces that thunk.

Same rule for and / or

`false and LOOP → false`

`true or LOOP → true`

Ordinary target application stays strict.

u and v are fresh, unused parameters

Predecessor carries a two-number state.

(0, 0) → (0, 1) → (1, 2) → (2, 3)

State transition

```
(previous, current)
   → (current, current + 1)
```

After n steps: (max(n − 1, 0), n).

Derived operations

Predecessor: first projection.
Subtraction: repeat predecessor.
Addition / multiplication: iteration.

The pair, successor, iteration, and projection all execute as pure beta reductions.

Build larger operations from closed terms.

```
LE m n = ZERO (SUB m n)
LT m n = NOT (LE n m)
GE m n = LE n m
GT m n = NOT (LE m n)
EQ m n = AND (LE m n)
             (LE n m)
```

Two different obligations

Library boolean combinators work on already evaluated values.

Source and / or also preserve short-circuit evaluation through delayed branches.

Tests cover equal and unequal operands, both comparison outcomes, and complete boolean truth tables.

Z delays recursive self-application.

```
Z = λg. (λx. g (λv. x x v))
        (λx. g (λv. x x v))

C[fix f(x). e] = Z (λf. λx. C[e])
```

> The extra λv makes the recursive argument a value.
> Recursion unfolds when the function is called.

Ordinary Y unfolds eagerly under call-by-value.
A recursive function itself completes as a lambda before application.

Observe behavior through pure target contexts.

Naturals

```
value STEP BASE

STEP = λtail c z. c tail
BASE = λc z. z
```

Reduce to an inert spine;
then count its nodes.

Booleans and pairs

Boolean: select one of two distinct closed markers.

Pair: evaluate projections, then observe each component.

Kind is explicit: Church zero and false have the same representation.
Computed values may contain redexes beneath lambdas.

2 + 3 × 4 evaluates to 14.

Source AST
2 + (3 * 4)

→

Pure term
ADD N(2) (MUL N(3) N(4))

→

Observed natural
14

Execution

4 beta steps

Budget: 20,000

Observation

25 beta steps

Separate budget: 10,000

Source result, explicit expected answer, and target observation agree.
The saved target file expands every abbreviation shown here.

The same pipeline handles recursion.

Program	Expected / observed	Execution	Observation
factorial(4)	**24 / 24**	471 steps	218 steps
Fibonacci(5)	**5 / 5**	1,794 steps	45 steps

```
fact(n) = if n == 0 then 1 else n * fact(n - 1)
fib(n)  = if n <= 1 then n else fib(n - 1) + fib(n - 2)
```

Both use Z, Church arithmetic, and delayed conditionals.
Neither answer is precomputed by the compiler.

Skipping is selective; application stays strict.

Source context	Target outcome
`if true then 1 else LOOP`	1
`false and LOOP / true or LOOP`	false / true
`(λx. 0) LOOP`	Budget exhausted
`let x = LOOP in 2`	Budget exhausted
`fst (pair 2 LOOP)`	Budget exhausted

LOOP also compiles successfully.
Only execution uses a finite reduction budget.

Test obligations, not just examples.

Automated coverage

49 unittest methods

Plus parameterized grids and edge cases.

14 / 14 required demos

What the checks establish

- Capture avoidance and free variables
- Budgets, traversal order, immutability
- Every operator and short-circuit case
- Pure output; no evaluator in compilation
- Standalone submission packaging

Each terminating case checks an explicit answer and the shared source result.
TESTING.md maps these checks to the specification.

Semantic evidence with explicit limits.

> For closed, well-typed programs returning naturals, booleans, or finite data pairs, the compiled observation agrees with the source result, given sufficient resources.

Why the claim is plausible

Local translation rules, capture avoidance, strict application, selected thunks, and delayed recursive calls.

What remains outside the claim

Finite tests are not a proof.
Substitution can grow terms.
Observers are not universal validators.
Budget exhaustion does not prove divergence.

Run the complete pipeline.

From assigns/06/ · Python 3.12 or later · no runtime dependencies

```
python -B -m unittest discover -s MySolution/TEST -t . -v
```

```
python -B -m MySolution.demo
```

Design rules: DESIGN.md
Coverage and measurements: TESTING.md + results/

Sources: Assign06.md, REQUIREMENTS.md, the unchanged course AST,
and attributed Assignment 5 scope visitors. AI assistance is documented.